
Laboratoire 5

IP AXI4-Lite avec FPGA I/O

Départements : TIC
Unité d'enseignement SCF

Auteur : Urs Behrmann
Professeurs : Alberto Dassatti
Yann Thomas
Assistant : Clément Dieperink
Classe : SCF
Salle de labo : A07
Date : 01.04.2026

Table des matières

1	Introduction	3
2	Ajout de l'IP AXI4-Lite dans Platform Designer	3
3	Plan d'adresse de l'IP	5
4	Programme C	5
5	Difficultés rencontrées	6
6	Tests sur la carte	6
7	Utilisation d'outil IA	6

1 Introduction

Ce rapport décrit le laboratoire 5 : une IP avec interface AXI4-Lite sur le bus léger HPS-FPGA, reliée aux entrées/sorties de la carte DE1-SoC (boutons, switches, LEDs, afficheurs 7 segments).

2 Ajout de l'IP AXI4-Lite dans Platform Designer

J'ai créé un nouveau composant dans le Component Editor à partir du fichier `axi4lite_slave.vhd`. Il faut lier les signaux du fichier VHDL aux interfaces attendues par Qsys : AXI4-Lite esclave, horloge, reset, et conduit pour les entrées/sorties vers le FPGA.

La figure 1 montre le symbole du bloc de l'IP. Les figures 2 et 3 montrent les paramètres (largeur des bus) et le placement des signaux dans les interfaces. La figure 4 montre le système `qsys_system` avec le HPS, l'horloge, et l'instance `AXI4Lite_0` connectée au maître léger AXI.

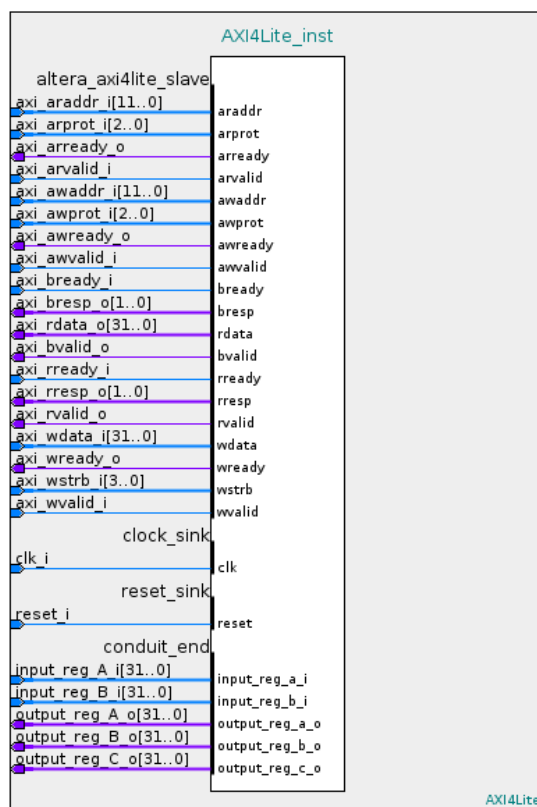


Figure 1: Symbole du composant AXI4-Lite.

	Name	Default V...	Edita...	Type	Group	Tooltip
Parameters:	AXI_DATA_WIDTH	32	☒	integer		
	AXI_ADDR_WIDTH	12	☒	integer		

Figure 2: Paramètres de l'IP (largeur d'adresse et de données).

```

▶ altera_axi4lite_slave AXI4Lite Slave
  ▶ axi_araddr_i [12] araddr
  ▶ axi_arprot_i [3] arprot
  ◀ axi_arready_o [1] arready
  ▶ axi_arvalid_i [1] arvalid
  ▶ axi_awaddr_i [12] awaddr
  ▶ axi_awprot_i [3] awprot
  ◀ axi_awready_o [1] awready
  ▶ axi_awvalid_i [1] awvalid
  ▶ axi_bready_i [1] bready
  ◀ axi_bresp_o [2] bresp
  ◀ axi_bvalid_o [1] bvalid
  ◀ axi_rdata_o [32] rdata
  ▶ axi_rready_i [1] rready
  ◀ axi_rresp_o [2] rresp
  ◀ axi_rvalid_o [1] rvalid
  ▶ axi_wdata_i [32] wdata
  ◀ axi_wready_o [1] wready
  ▶ axi_wstrb_i [4] wstrb
  ▶ axi_wvalid_i [1] wvalid
▶ clock_sink Clock Input
  ▶ clk_i [1] clk
▶ conduit_end Conduit
  ▶ input_reg_A_i [32] input_reg_a_i
  ▶ input_reg_B_i [32] input_reg_b_i
  ◀ output_reg_A_o [32] output_reg_a
  ◀ output_reg_B_o [32] output_reg_b
  ◀ output_reg_C_o [32] output_reg_c
  <<add signal>>
▶ reset_sink Reset Input
  ▶ reset_i [1] reset
  <<add signal>>
<<add interface>>

```

Figure 3: Signaux et interfaces (AXI4-Lite, horloge, reset, conduit).

System: qsys_system Path: hps_0							
Use	Connections	Name	Description	Export	Clock	Base	End
☒		hps_0	Arria V/Cyclone V Hard Proce...	memory			
		memory	Conduit	hps_io_0			
		hps_io	Conduit				
		h2f_reset	Reset Output				
		h2f_lw_axi_clock	Clock Input		clk_0		
		h2f_lw_axi_master	AXI Master				
☒		clk_0	Clock Source				
		clk_in	Clock Input	clk			
		clk_in_reset	Reset Input				
		clk	Clock Output		exported		
		clk_reset	Reset Output				
☒		AXI4Lite_0	AXI4Lite				
		altera_axi4lite_slave	AXI4Lite Slave				
		clock_sink	Clock Input				
		reset_sink	Reset Input				
		conduit_end	Conduit				

Figure 4: Contenu du système Qsys : connexions et adressage.

Après chaque modification du VHDL ou du composant, j'ai régénéré les fichiers HDL du système, puis recompilé le projet Quartus. Le fichier `DE1_SoC.qsf` inclut le `.qip` généré et le fichier `DE1_SoC_top.vhd` relie les signaux du conduit aux broches de la carte (keys, SW, LEDR, HEX).

3 Plan d'adresse de l'IP

Les registres sont alignés sur 32 bits. Les adresses sont des *décalages* par rapport à la base du périphérique sur le bus (souvent `0xFF200000` pour le pont léger).

Décalage	Rôle	Lecture / écriture
0x00	Constante 0xBADB100D (identification)	R
0x04	Registre de test	RW
0x08	État des boutons (keys)	R
0x0C	Capture de fronts sur les boutons	RW
0x10	État des switches	R
0x14	LEDs (sortie)	RW
0x18	Set LEDs (écrire 1 met le bit à 1)	W
0x1C	Clear LEDs (écrire 1 met le bit à 0)	W
0x20	Afficheurs HEX3 à HEX0	RW
0x24	Afficheurs HEX5 et HEX4	RW

4 Programme C

Le programme lit d'abord la constante et compare avec la valeur attendue. Il écrit une valeur dans le registre de test et la relit pour vérifier que les accès mémoire fonctionnent.

Ensuite, dans une boucle infinie, il utilise le registre de capture de fronts sur les boutons : chaque action sur une touche déclenche une petite logique (valeur interne liée aux switches, incrément, décrétement, remise à zéro). Selon si l'opération est "légale" ou non, il allume ou éteint les LEDs avec les registres set/clear. Il affiche la valeur sur les afficheurs 7 segments.

5 Difficultés rencontrées

J'ai eu beaucoup de difficultés avec l'IP au début. Dans le fichier TCL du Component Editor, les noms des ports pour l'interface AXI4-Lite ne correspondaient pas exactement aux noms dans mon entity VHDL (`clk_i`, `axi_awaddr_i`, etc.). Du coup, la synthèse et l'analyse Quartus échouaient avec des erreurs du type "port inexistant".

J'ai dû aligner les noms dans le TCL sur le VHDL, puis régénérer plusieurs fois les fichiers Qsys et refaire la compilation jusqu'à ce que tout soit cohérent. C'était long mais ça m'a appris à bien vérifier la correspondance entre Platform Designer et le code RTL.

6 Tests sur la carte

J'ai programmé le FPGA avec le fichier `.sof`, chargé le preloader pour le HPS, puis lancé le programme depuis Arm Development Studio. J'ai vérifié sur la carte Cyclone V (DE1-SoC) que les affichages `printf` apparaissaient, que les LEDs et les HEX réagissaient aux boutons et aux switches, et que le comportement correspondait au cahier des charges du laboratoire.

7 Utilisation d'outil IA

Pour ce rapport, j'ai utilisé l'assistant *Claude Sonnet* pour corriger la grammaire et l'orthographe du rapport. Le fond technique reste le mien ; seule la forme du français a été améliorée.